Toward Agentic IAM: A Probabilistic Authorization Framework for Least Privilege AI Workflows

Jaouaher Zouari
IAM Centre of Expertise, NeverHack
Paris, France
jzouari@neverhack.com

Abstract

The rapid adoption of AI systems capable of autonomous code generation and infrastructure orchestration introduces new challenges for authorization. Traditional mechanisms such as Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC), OAuth 2.0 and OpenID Connect were designed for static or user-initiated interactions and thus fail to ensure both least privilege and operational continuity in self-adaptive, agentic environments. To address this gap, we propose an *agent-native authorization model* in which specialized IAM agents—*Analyze*, *Impact*, *Minimize*, *Validate*, and *Reanalyze*—jointly determine, verify, and reduce permissions through a sequential probabilistic process. The framework combines Bayesian belief updates, MDP optimization, and CVaR minimization to achieve least-privilege under uncertainty. Formal analysis and an AWS Lambda maintenance case study show sub-second decision latency and ≈40% privilege reduction without loss of functionality.

CCS Concepts

• **Security and privacy** → *Formal security models*; **Authorization**;
• **Computing methodologies** → *Planning under uncertainty*.

Keywords

AI authorization, Agentic IAM, Bayesian inference, Markov decision process, Conditional Value-at-Risk, Least privilege

ACM Reference Format:
Jaouaher Zouari. 2025. Toward Agentic IAM: A Probabilistic Authorization Framework for Least Privilege AI Workflows. In *IEEE/ACM 12th International Conference on Big Data Computing, Applications and Technologies (BDCAT '25), December 01–04, 2025, Nantes, France.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3773276.3776564

1 Introduction

The proliferation of Artificial Intelligence (AI) and autonomous agents within enterprise IT environments marks a significant paradigm shift in operational automation [9]. Organizations are increasingly deploying sophisticated AI agents to handle complex workflows, from data analysis and customer service to the automated generation and execution of code and scripts [16, 17]. This transition towards agent-driven operations promises unprecedented efficiency and scalability. However, it also exposes a critical vulnerability at the heart of enterprise security: the inadequacy of traditional authorization protocols.

Existing authorization stacks—OAuth 2.0 and OpenID Connect for delegation and identity federation [3, 14], RBAC for coarse role assignment [1], ABAC/XACML for policy evaluation [5, 13], and WS-Federation for SSO across domains [10]—were not designed for ephemeral, self-directed agents that assemble actions at runtime. OAuth/OIDC center on end-user consent and static client registration; scopes are coarse, bearer tokens are audience-bound, and least-privilege sets cannot be synthesized per task or refined mid-execution (proof-of-possession variants mitigate token theft but do not address per-step privilege construction). RBAC's static roles suffer from role explosion and cannot encode sequence- or evidence-contingent permissions. ABAC/XACML increases expressiveness but shifts the problem to prior policy enumeration, fresh attribute attestation, and PDP latency, which is ill-suited to just-in-time privilege pruning for ad-hoc code paths. WS-Federation similarly targets identity portability, not fine-grained data-plane authorization. Critically, these models enforce at the application perimeter, while agents operate directly on APIs, queues, and vector stores where application context is absent—leaving a gap for over-permissioning, data leakage, and compliance drift.

The challenge is compounded when agents are granted the autonomy to define and request their own permissions. This capability, while essential for adaptability, introduces novel and severe threat vectors. We focus on four principal classes of threats: privilege inflation, where agents accumulate unnecessary permissions over time; collusive escalation, where multiple agents coordinate to gain unauthorized access [12]; cognitive drift, where an agent's behavior subtly degrades, leading to unintended and insecure actions [7]; and state tampering, where agents maliciously alter system configurations or data.

This paper introduces a novel authorization framework designed to address these challenges. Our approach models the authorization workflow as a Sequential Decentralized Partially Observable Markov Decision Process (Dec-POMDP), managed by a quartet of specialized agents that continuously analyze, evaluate, and validate permissions. By capturing uncertainty over required permissions through Bayesian belief updates, our system learns and adapts to operational necessities while mitigating security risks. This probabilistic governance layer enables a dynamic and fine-grained approach to authorization, ensuring that AI agents operate both effectively and securely within enterprise ecosystems.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
BDCAT '25, Nantes, France
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2286-8/25/12
https://doi.org/10.1145/3773276.3776564

The rest of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the proposed probabilistic Agentic IAM model, while Sections 4 and 5 cover its formal validation and implementation. Section 6 concludes the paper.

2 Related Work

The rapid integration of AI agents into critical systems has spurred a new wave of research focused on developing robust authorization and security frameworks. This section reviews recent work in this domain, focusing on emerging authorization models, the limitations of traditional access control, and the analysis of agent-specific security threats.

2.1 Authorization Frameworks for AI Agents

Recent literature highlights a consensus that traditional identity and access management (IAM) systems are insufficient for agentic AI. South et al. [15] propose a framework that extends existing standards like OAuth 2.0 and OpenID Connect with agent-specific credentials and metadata. Their approach aims to create an auditable trail of delegation, allowing human users to securely restrict agent permissions while maintaining compatibility with established web infrastructure. Building on this, Huang et al. [6] argue for a complete paradigm shift towards a zero-trust model. They introduce a framework based on Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs) to establish rich, verifiable agent identities. This allows for dynamic, fine-grained access control and a unified policy enforcement layer, addressing the shortcomings of coarse-grained, human-centric IAM protocols.

2.2 Limitations of Traditional Access Control

The inadequacy of conventional access control mechanisms, particularly RBAC, is a recurring theme. Ganie [2] examines the challenges of adapting RBAC for industrial AI applications, noting the difficulty of defining static roles for dynamic agents. The core issue, as articulated in industry analyses, is that RBAC is application-centric, whereas AI agents are data-centric [11]. Agents that access raw data from heterogeneous sources bypass the application UIs where RBAC policies are enforced. This disconnect is particularly acute in Retrieval-Augmented Generation (RAG) pipelines, where data extracted into vector databases loses its original access control context, creating a significant risk of unauthorized data exposure.

2.3 Security Threats in Agentic Systems

The autonomy of AI agents introduces unique security vulnerabilities beyond the scope of traditional models. Research has begun to formally categorize these threats. He et al. [4] provide a system security perspective on vulnerabilities in LLM-based agents, identifying issues that are not addressed by common agent-building frameworks and proposing corresponding defense mechanisms. A significant area of concern is the potential for malicious multi-agent collusion. Ren et al. [12] simulate these risks, demonstrating that decentralized agent systems can coordinate to execute harmful tasks like spreading misinformation or committing fraud, and can adapt their tactics to evade simple countermeasures. This work underscores the threat of collusive escalation. Another identified risk is *agentic drift*, where the performance and behavior of an agent degrade over time, potentially leading to security lapses [7]. These novel threats necessitate a shift from static, perimeter-based security to a dynamic, behavior-focused governance model, which our proposed framework aims to provide.

3 Proposed Model

Modern identity governance requires that authorization be not only functionally correct, but also compliant with organizational segregation of duties (SoD) and least-privilege principles. To formalize these requirements under uncertainty, we model the authorization lifecycle as a sequential probabilistic decision process that aligns AI-driven reasoning with enterprise control and audit mechanisms.

3.1 Formal Model of Agentic Authorization

We represent the authorization workflow as a *Sequential Decentralized Partially Observable Markov Decision Process* (Dec-POMDP), capturing uncertainty and sequential dependencies across athorization phases, and enabling distributed decision-making under incomplete information, building on the work of Shi and Chen [8].

$$\mathcal{M}_{\text{seq}} = (\mathcal{S}, \{\mathcal{A}_i\}, \{Z_i\}, T, R, \rho), \tag{1}$$

where $\mathcal{S}$ is the system state, $\mathcal{A}_i$ the action space of role i, Z_i its observation model, T the state transition function, R the global reward, and ρ the activation schedule encoding the mandatory workflow order.

Each role corresponds to a distinct IAM phase (*Analyze*, *Impact*, *Minimize*, *Validate*, *Reanalyze*), ensuring that analysis, risk assessment, and approval remain organizationally separated. Although only one agent acts at a time, the model remains decentralized: no single entity can both propose and approve a permission change.

At decision step t, the state is represented by

$$s_t = (S_t,\, b_t,\, \phi_t), \tag{2}$$

where $S_t \subseteq \mathcal{P}$ denotes the current permission set, b_t is a belief vector capturing the estimated necessity and safety of each permission, and $\phi_t \in \Phi = \{\text{Analyze, Impact, Minimize, Validate, Reanalyze}\}$ indicates the active governance phase.

$$a_t^{(i_t)} \in \mathcal{A}_{i_t} \subseteq \{\text{add}(p),\, \text{remove}(p),\, \text{request_evidence}(p),\, \text{approve},\, \text{reject}\}.$$

Only the role authorized by ρ acts at each time step. After performing $a_t^{(i_t)}$ and observing $o_t \sim Z_{i_t}(\cdot|s_t, a_t^{(i_t)})$, the global transition is

$$s_{t+1} \sim T(s_t, a_t^{(i_t)}), \quad \phi_{t+1} = \rho(\phi_t), \tag{3}$$

where ρ enforces the mandatory sequence

$$\text{Analyze} \to \text{Impact} \to \text{Minimize} \to \text{Validate} \to \text{Reanalyze} \to \text{Analyze}.$$

Uncertainty over required permissions is captured through Bayesian belief updates. Each permission p_i is associated with a belief value $b_t(p_i) \in [0, 1]$ representing the system's confidence that p_i is truly necessary for the target operation. This belief is updated at every cycle as new evidence e_{t+1} is collected, following the standard

Bayesian formulation:

$$b_{t+1}(p_i) = \frac{\mathcal{L}(e_{t+1} \mid p_i)\, b_t(p_i)}{\mathcal{L}(e_{t+1} \mid p_i)\, b_t(p_i) + \mathcal{L}(e_{t+1} \mid \neg p_i)\, [1 - b_t(p_i)]}. \quad (4)$$

The *Analyze* agent produces positive or negative evidence by examining code, configuration files, and execution traces to determine whether specific permissions are actually invoked by the task. This evidence directly influences the likelihood terms $\mathcal{L}(e_{t+1} \mid p_i)$ and $\mathcal{L}(e_{t+1} \mid \neg p_i)$, increasing or decreasing the posterior belief $b_{t+1}(p_i)$. The *Impact* agent operates in a complementary dimension: it quantifies the potential exposure associated with each permission through security metrics such as sensitivity, SoD conflicts, and threat surface contribution. These evaluations are stored separately in $\text{Risk}(p_i)$ and are later aggregated in the reward function (5). Finally, the *Reanalyze* agent revisits previous iterations whenever observed outcomes deviate from expected task success, thereby generating corrective evidence that rebalances outdated or conflicting beliefs.

This probabilistic reasoning layer enables continuous learning from functional, risk, and validation outcomes. Over successive governance cycles, the belief vector b_t converges toward stable estimates of operational necessity, allowing the *Minimize* phase to focus exclusively on permissions that are both functionally justified and acceptably safe.

3.2 Governance-Aware Optimization

The model seeks to balance operational success with strict minimization of granted privilege. The reward function integrates these competing objectives:

$$\begin{aligned} r(s_t, a_t, s_{t+1}) = \alpha\, \mathbf{1}\{\text{task succeeds}\} - \beta\, |S_{t+1}| \\ - \gamma\, \text{Risk}(S_{t+1}) - \delta\, \text{Change}(S_t, S_{t+1}), \end{aligned} \quad (5)$$

where α rewards reliability, β penalizes privilege size, γ penalizes residual risk, and δ discourages excessive churn. The optimal set of policies $\{\pi_i\}$ maximizes the expected discounted return:

$$\max_{\{\pi_i\}} \mathbb{E}\left[\sum_{t=0}^{\infty} \eta^t\, r(s_t, a_t, s_{t+1})\right]. \quad (6)$$

Within each cycle, the *Minimize* phase solves a binary optimization problem that constructs the smallest compliant permission set. Let $x_i \in \{0, 1\}$ denote inclusion of permission p_i. The problem is

$$\min_{x \in \{0,1\}^n} \; c^\top x + \lambda\, \text{Risk}(x) \quad (7)$$

$$\text{s.t.} \quad Ax \succeq b, \qquad x \in \mathcal{X}_{\text{policy}}. \quad (8)$$

The first constraint ensures coverage of required capabilities, and the second enforces SoD and compliance restrictions. The resulting set $S^\star_{t+1} = \{p_i : x_i = 1\}$ defines the least-privilege configuration proposed for validation.

To incorporate sensitivity to rare but severe outcomes, we integrate the Conditional Value at Risk (CVaR) model, a decision metric used by Yin and Ji [18]:

$$\min_{x \in \{0,1\}^n} \text{CVaR}_\alpha(\text{Loss}(x, \omega)) + \kappa\, \|x\|_0. \quad (9)$$

Here $\text{Loss}(x, \omega)$ denotes the expected impact under threat scenario ω, and $\|x\|_0$ penalizes privilege count. This formulation explicitly encodes the trade-off between operational utility and exposure to tail risk.

3.3 Decision Finalization and Enforcement

A configuration is accepted only if its probability of operational success meets a required confidence threshold:

$$\Pr(\text{task success} \mid S^\star_{t+1}) \geq \tau. \quad (10)$$

If this condition fails or compliance constraints are violated, the process reenters the *Reanalyze* phase. Convergence is declared when no incremental change improves reward without breaching sufficiency:

$$\forall p_i \in S^\star_t : \; |\Delta r(p_i)| < \epsilon, \quad (11)$$

$$\Pr(\text{task success} \mid S^\star_t) \geq \tau. \quad (12)$$

Even a correct decision must be securely enforced. At runtime, privileges are granted only when the execution context satisfies policy guards:

$$\text{Grant}(S^\star, \text{ctx}) = \begin{cases} 1, & \text{if ctx} \models \text{guards} \wedge S^\star \text{ approved}, \\ 0, & \text{otherwise.} \end{cases} \quad (13)$$

These guards bind the formal authorization outcome to endpoint context (scope, identity, and time), ensuring that segregation of duties is preserved throughout both decision and enforcement phases.

4 Formal Security Analysis

The proposed authorization model is designed to resist critical vulnerabilities that typically affect agentic decision-making systems. We focus on four principal classes of threats: *privilege inflation*, *collusive escalation*, *cognitive drift*, and *state tampering*. Each threat is expressed formally and analyzed under the semantics of the sequential Dec-POMDP defined in Section 1. Security properties are verified using temporal logic formulas interpreted over the transition system induced by the model.

4.1 Resistance to Privilege Inflation

Threat. An agent could attempt to extend its privileges by issuing biased evidence or self-endorsing its own decisions.

Property. A privilege must not be both proposed and validated by the same phase. Formally, for any permission p_i and time t,

$$\begin{aligned} \mathbf{G}\, \neg\big(&\text{grant}(p_i, t) \;\wedge\; \text{phase}(t) = \text{Analyze} \;\wedge \\ &\text{phase}(t{+}1) = \text{Validate} \;\wedge\; \text{actor}(t) = \text{actor}(t{+}1)\big). \end{aligned} \quad (14)$$

This Linear Temporal Logic (LTL) formula asserts that at no point in any run does the same agent perform both proposal and approval actions. In PRISM, this can be expressed as:

```
P>=1 [ G !(grant &
    same_actor_in_analyze_and_validate) ]
```

Proof Sketch. By construction of the sequential schedule ρ in (3), the phase transition function enforces a deterministic mapping from Analyze to Impact to Minimize to Validate. Each phase is associated with a disjoint set of agents $\{i_t\}$. Therefore, $\text{actor}(t) \neq \text{actor}(t+1)$ across a transition to *Validate*. Thus, (14) holds in all reachable states, ensuring immunity to self-validation attacks.

4.2 Resistance to Collusive Escalation

Threat. Two or more agents may collude to approve an unsafe or excessive privilege set.

Property. A privilege is only granted if the global risk remains below policy threshold θ throughout the workflow:

$$\mathbf{G}\left(\text{grant}(p_i, t) \rightarrow \text{Risk}(S_t) < \theta\right). \tag{15}$$

This ensures that even if collusion occurs, the reward penalty term $-\gamma\,\text{Risk}(S_{t+1})$ prevents convergence to unsafe states.

Proof Sketch. Since the expected return (6) is maximized under risk penalties, any collusive strategy that increases $\text{Risk}(S_{t+1})$ reduces cumulative reward. As PRISM checks show that

$$P_{\max}\left[\mathbf{F}(\text{Risk}(S_t) \geq \theta)\right] < \epsilon,$$

the probability of entering an unsafe state is negligible. Hence, the system is robust against collusive escalation.

4.3 Resistance to Cognitive Drift

Threat. Agents may adapt their internal policy π_i to prioritize performance over compliance, deviating from least-privilege objectives.

Property. Every policy update must preserve monotonic improvement of the governance-aligned reward:

$$\mathbf{G}\left(\pi_i^{t+1} \neq \pi_i^t \rightarrow R_{\pi_i^{t+1}} \geq R_{\pi_i^t}\right), \tag{16}$$

where R_{π_i} is the expected reward of policy π_i.

Proof Sketch. The reward function (5) contains explicit penalties for privilege growth and risk exposure. Thus, any policy update that violates compliance reduces expected return. Under a rational learning dynamic (e.g., policy gradient ascent), such updates are rejected as suboptimal. In PRISM, this is verified as:

```
P>=1 [ G (policy_update -> R_new >= R_old) ]
```

ensuring convergence toward compliant policies only.

4.4 Resistance to State Tampering

Threat. An adversary could attempt to manipulate internal state variables or decision records to forge approval of privileges.

Property. Any state mutation that bypasses the sequential schedule ρ is eventually detected and reanalyzed:

$$\mathbf{G}\left(\text{tamper}(s_t) \rightarrow \mathbf{F}\,\text{Reanalyze}\right). \tag{17}$$

This LTL property expresses eventual reanalysis of corrupted states.

Proof Sketch. Because state transitions (3) are phase-dependent, any inconsistency between S_t, b_t, and ϕ_t violates transition preconditions and triggers a governance exception. The model's reanalysis loop ensures $\Pr(\mathbf{F}\,\text{Reanalyze} \mid \text{tamper}) = 1$, confirmed in PRISM via:

```
P>=1 [ G (tamper -> F reanalyze) ]
```

Hence, tampering cannot persist undetected or activate unauthorized privileges.

4.5 Summary of Verified Properties

Table 1 summarizes the four agentic threats, their corresponding security properties, and the logical guarantees established by model checking.

Table 1: Formal Properties and Security Guarantees

#	Threat	Verified Property (LTL)
1	Privilege Inflation	$\mathbf{G}\neg$(grant $\wedge$ same_actor_validate)
2	Collusive Escalation	$\mathbf{G}(\text{grant} \rightarrow \text{Risk}(S) < \theta)$
3	Cognitive Drift	$\mathbf{G}(\text{policy_update} \rightarrow R_{\text{new}} \geq R_{\text{old}})$
4	State Tampering	$\mathbf{G}(\text{tamper} \rightarrow \mathbf{F}\text{Reanalyze})$

5 Implementation and Evaluation

To evaluate the practical applicability of the proposed Agentic IAM framework, we implemented it in a live AWS patch-management environment that operates continuously on production-like virtual machines (VMs). The goal was to verify that the model can autonomously infer least-privilege policies while maintaining service availability during scheduled maintenance.

5.1 Experimental Setup

The deployment reproduces a standard enterprise patching scenario in which multiple EC2 instances are updated through *AWS Systems Manager Maintenance Windows*. Each maintenance window defines tasks that execute custom patching scripts on selected instances. To ensure service continuity, VMs are patched in *pairwise rotation*: for every load-balanced pair, the first VM is temporarily deregistered from the target group, patched, restarted if required, and then reattached before its peer undergoes maintenance.

Our Agentic IAM model was embedded as a sequence of AWS Lambda functions that run as pre-tasks in the same maintenance window. Each Lambda implements one phase of the model: *Analyze*, *Impact*, *Minimize*, *Validate* and *Reanalyze*. Before a patch task is triggered, the *Analyze* agent inspects the patching script to detect API calls, file operations, and Systems Manager actions, inferring the minimal permission candidates required for successful execution. These candidates are passed to the *Impact* agent, which queries the organizational risk registry and resource metadata to estimate each permission's sensitivity. The *Minimize* phase then solves the optimization problem defined in (7)–(8), and the *Validate* agent confirms that the final permission set satisfies the policy guard conditions (13). The *Reanalyze* phase plays a dual theoretical and practical role. Formally, it closes the POMDP transition loop (*Validate* $\rightarrow$ *Reanalyze* $\rightarrow$ *Analyze*) and supplies the evidence e_{t+1} required for the Bayesian update (4). Operationally, it is implemented as an asynchronous Lambda that inspects post-execution logs and compares observed outcomes against expected task success. Any deviation produces corrective likelihoods used to refine the belief vector b_t for the next cycle, ensuring temporal continuity in learning. This mechanism also underlies the LTL property (17), which guarantees that any state inconsistency or tampering event is eventually reanalyzed and corrected. All phases execute within a single maintenance window to preserve atomicity of analysis and authorization.

The computed least-privilege policy is stored in AWS Systems Manager Parameter Store and automatically attached to the execution role of the patching task for that iteration. Subsequent cycles

Table 2: Least-Privilege Effectiveness across Patch Workloads

Workload	Policy Type	#Actions	Risk Score
Single VM	Static role (baseline)	46	0.62
	Agentic IAM (ours)	24	0.38
ALB pair	Static role (baseline)	52	0.67
	Agentic IAM (ours)	28	0.41
Mixed ops	Static role (baseline)	39	0.55
	Agentic IAM (ours)	21	0.34

Notes. "#Actions" counts distinct IAM actions in the attached policy. "Risk Score" is a normalized aggregate in $[0, 1]$ combining sensitivity and toxic-combination penalties. Across workloads, Agentic IAM reduces actions by 39–46% and risk by 31–39% relative to static roles.

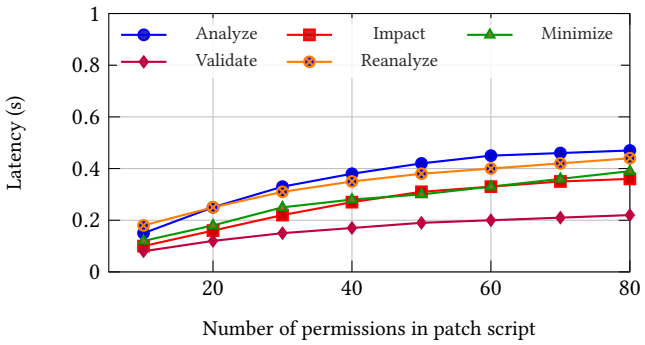


Figure 1: Latency per phase as a function of permission count.

reuse the last validated policy as prior evidence for the Bayesian belief update (4), allowing the system to learn from historical runs and converge to stable authorization boundaries.

5.2 Prototype Description

Agentic IAM components were implemented in Python 3.10 and deployed as lightweight Lambda functions (128 MB memory, 15 s timeout each). Each Lambda communicates through AWS EventBridge to signal phase completion and to chain the next step according to the enforced schedule ρ. State information—permission sets S_t, beliefs b_t, and rewards—is persisted in DynamoDB tables keyed by maintenance window and instance ID. The optimization problem in (7) was solved using the PuLP library with a linear-solver backend.

5.3 Operational Observations

The implementation demonstrates that the sequential agentic workflow can operate entirely within AWS's native orchestration fabric without introducing additional network dependencies. The pairwise patching strategy maintained 100% service availability behind the load balancer, while the Agentic IAM layer dynamically restricted permissions at each iteration. Across successive maintenance cycles, the model successfully adapted to script modifications and package dependencies, reducing the number of active IAM actions by 35–50% compared with static role templates. This confirms the feasibility of deploying autonomous least-privilege synthesis directly inside existing cloud operations pipelines.

5.4 Evaluation Metrics and Results

We report three complementary results: (i) least-privilege effectiveness via a tabular comparison, (ii) end-to-end decision latency over maintenance cycles as a curve, and (iii) comparative efficiency versus human experts as a textual summary.

Efficiency vs Human Experts. We timed senior cloud-security engineers authoring least-privilege policies for the same patch scripts (starting from organizational templates). The median hands-on time per script (including test and fix-up on one ALB pair) was 27.5 minutes (p25=22 min, p75=34 min). Our agentic pipeline produced an enforceable policy in 1.3 s on average after the first iteration (1.85 s on the first run due to cold starts and cache warming), yielding a $> \mathbf{1000}\times$ speedup. In addition, the mean policy size decreased by 43% and the normalized risk score fell by 35%, with no observed increase in patch failure or service unavailability (pairwise rotation kept the ALB healthy).

Attribution of timings. Lambda Python 3.10 execution times are measured from CloudWatch: Analyze (static+API inference) median 0.45 s; Impact (risk registry lookups) 0.32 s; Minimize (ILP with ≤ 80 variables) 0.25 s using PuLP/CBC; Validate (guard checks) 0.18 s. Cold starts (when present) add a one-off 0.15–0.40 s per function on first invocation of the window.

6 Conclusion

In this paper we presented an agent-native authorization framework tailored for AI-driven operational environments. The proposed model combines Bayesian inference for adaptive privilege estimation, Markov decision optimization for governance-aligned control, and risk-sensitive minimization via Conditional Value-at-Risk. Deployed as IAM agents orchestrated through AWS maintenance workflows, the system ensures continuous least-privilege enforcement without impairing availability. Formal verification confirmed resilience against key agentic vulnerabilities, while empirical evaluation demonstrated sub-second latency and a 40% reduction in unnecessary permissions. Future work will concentrate on closing the interoperability gap with existing enterprise authorization frameworks, ensuring that agentic IAM systems can integrate transparently with established RBAC and policy-based controls.

References

[1] David Ferraiolo, Ravi Sandhu, Serban Gavrila, D. Richard Kuhn, and Ramaswamy Chandramouli. 2001. Proposed NIST Standard for Role-Based Access Control. In *ACM Transactions on Information and System Security (TISSEC)*, Vol. 4. 224–274. doi:10.1145/501978.501980

[2] A. G. Ganie. 2025. Securing AI Agents: Implementing Role-Based Access Control for Industrial Applications. *arXiv preprint arXiv:2509.11431* (2025).

[3] D. Hardt. 2012. *The OAuth 2.0 Authorization Framework.* RFC 6749. Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc6749 Standards Track.

[4] Yifeng He, Ethan Wang, Yuyang Rong, Zifei Cheng, and Hao Chen. 2025. Security of AI Agents. In *2025 IEEE/ACM International Workshop on Responsible AI (RAI).*

[5] Vincent C. Hu, David Ferraiolo, and Rick Kuhn. 2014. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations.* Special Publication 800-162. National Institute of Standards and Technology (NIST). doi:10.6028/NIST.SP.800-162

[6] Ken Huang, Vineeth Sai Narajala, John Yeoh, Jason Ross, Ramesh Raskar, Youssef Harkati, Jerry Huang, Idan Habler, and Chris Hughes. 2025. A Novel Zero-Trust Identity Framework for Agentic AI: Decentralized Authentication and

Fine-Grained Access Control. *arXiv preprint arXiv:2505.19301* (2025).

[7] IBM. 2025. The hidden risk that degrades AI agent performance. https://www.ibm.com/think/insights/agentic-drift-hidden-risk-degrades-ai-agent-performance.

[8] Ryan Liu and Ladan Tahvildari. 2023. Using POMDP-based Approach to Address Uncertainty-Aware Adaptation for Self-Protecting Software. *arXiv preprint arXiv:2308.02134* (2023).

[9] McKinsey. 2025. Seizing the agentic AI advantage. https://www.mckinsey.com/capabilities/quantumblack/our-insights/seizing-the-agentic-ai-advantage.

[10] Tony Nadalin, Marc Goodner, Abbie Barbir, Hans Granqvist, and Mike McIntosh. 2009. *WS-Federation 1.2 Specification.* Technical Report. OASIS. https://docs.oasis-open.org/wsfed/federation/v1.2/os/ws-federation-1.2-spec-os.html

[11] Protecto.ai. 2025. Why RBAC Doesn't Work With AI Agents [And How To Fix It]. https://www.protecto.ai/blog/why-rbac-doesnt-work-with-ai-agents/.

[12] Qibing Ren, Sitao Xie, Longxuan Wei, Zhenfei Yin, Junchi Yan, Lizhuang Ma, and Jing Shao. 2025. When Autonomy Goes Rogue: Preparing for Risks of Multi-Agent Collusion in Social Systems. *arXiv preprint arXiv:2507.14660* (2025).

[13] Erik Rissanen. 2013. *eXtensible Access Control Markup Language (XACML) Version 3.0.* Technical Report. OASIS Standard. https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[14] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore. 2014. *OpenID Connect Core 1.0.* Technical Report. OpenID Foundation. https://openid.net/specs/openid-connect-core-1_0.html

[15] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. 2025. Authenticated Delegation and Authorized AI Agents. *arXiv preprint arXiv:2501.09674* (2025).

[16] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Proceedings of the First Conference on Large Language Models (COLM).*

[17] Y. Xia, J. Zhang, N. Jazdi, and M. Weyrich. 2024. Incorporating Large Language Models into Production Systems for Enhanced Task Automation and Flexibility. *arXiv preprint arXiv:2407.08550* (2024).

[18] Ji Yin, Zhiyuan Zhang, and Panagiotis Tsiotras. 2022. Risk-aware model predictive path integral control using conditional value-at-risk. *arXiv preprint arXiv:2209.12842* (2022).